

Integridade

Confidencialidade

Disponibilidade

ROTEIRO

Os 5 blocos conceituais que a lista insiste em cobrar

1. Ameaças e pilares

WannaCry, ransomware, bucket público, dupla extorsão e leitura correta da tríade CID.

2. Criptografia aplicada

TLS híbrido, assinatura digital, certificado, AC e escolha entre AES, hash e chaves públicas.

3. Vulnerabilidades web

SQL Injection, autenticação, manipulação de entrada e práticas de secure coding.

4. Identidade e autorização

JWT, RBAC, autenticidade, não repúdio e integridade de tokens.

5. DevSecOps e governança

Segredos em repositório, hooks, varredura de secrets, rotação de credenciais e resposta rápida.

WannaCry: classificar a ameaça e apontar o pilar primário afetado

Vulnerabilidade
sem patch

Propagação
autônoma

Criptografia
dos dados

Extorsão

Diagnóstico

- Ransomware pelo objetivo final: cifrar e extorquir.
- Worm pelo modo de propagação: replica-se automaticamente pela rede.
- Não depende de ação do usuário após a exploração inicial.
- Resposta forte: "wormable ransomware".

Alvo primário

Disponibilidade

Os dados existem, mas ficam indisponíveis ao usuário legítimo.

Infraestrutura

- Gestão de patches e atualização de sistemas expostos.
- Segmentação de rede para reduzir movimento lateral.
- EDR/antimalware e bloqueio de tráfego malicioso.

Governança

- Backups offline/testados com plano de recuperação.
- Gestão de vulnerabilidades e inventário de ativos.
- Resposta a incidentes com papéis e comunicação definidos.

HTTPS/TLS: por que a web moderna usa criptografia híbrida

Handshake em 5 atos

1 Client Hello

Cliente propõe versões, cifras e parâmetros.

2 Certificado

Servidor prova sua identidade com certificado assinado por AC.

3 Troca de chaves

Asimetria ou ECDHE protege o acordo da chave de sessão.

4 Sessão

Ambos derivam a mesma chave temporária.

5 Dados

AES/ChaCha20 protege o conteúdo em alta velocidade.

Por que "híbrida"?

- Criptografia assimétrica resolve autenticação e troca segura do segredo inicial.
- Criptografia simétrica assume depois porque é muito mais eficiente para tráfego massivo.
- Resultado: segurança sem transformar o navegador em uma torradeira em chamas.

Certificado digital + AC

- O certificado vincula o domínio à chave pública do servidor.
- A Autoridade Certificadora assina esse vínculo e entra na cadeia de confiança do navegador.
- Se a assinatura ou o domínio não fecham, o navegador acusa erro de autenticidade.

SQL Injection: quando a entrada do usuário vira comando do banco

Violação direta

Confidencialidade

A API retorna todos os prontuários, expondo dados médicos sensíveis a quem não deveria vê-los.

Efeito colateral

- Integridade também pode ser atingida se o atacante alterar ou apagar dados.
- Disponibilidade entra em risco se a exploração derrubar serviço ou banco.

Exemplo vulnerável

```
1 const q = "SELECT * FROM
  pacientes
2 WHERE nome = '" + busca + "'"
3 AND user_id = '" + userId +
  "'";
4 db.query(q);
```



Payload do atacante

'OR 1=1 --

O restante da query vira comentário; o filtro lógico deixa de restringir o resultado.

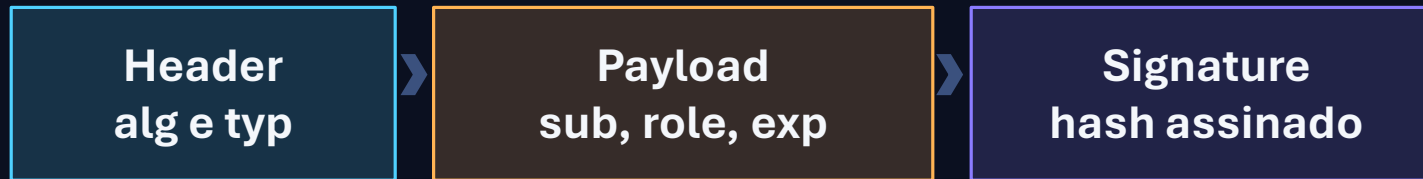
Correção

```
1 const sql = "SELECT * FROM
  pacientes
2 WHERE nome = $1 AND user_id =
  $2";
3 db.query(sql, [busca,
  userId]);
```

- Use prepared statements/queries parametrizadas.
- Valide entrada e aplique princípio do menor privilégio no banco.
- Nunca monte SQL por concatenação de strings.

JWT em microsserviços: integridade, autenticidade e o mito do "Base64 seguro"

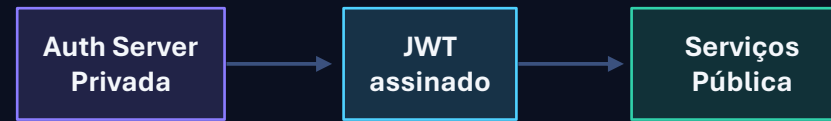
Estrutura do token



Confidencialidade x Integridade

- JWT comum é codificado, não criptografado: quem recebe pode ler o payload.
- Logo, não há confidencialidade por padrão para os claims.
- A assinatura protege a integridade: se alguém altera o payload, a verificação falha.

Fluxo assimétrico



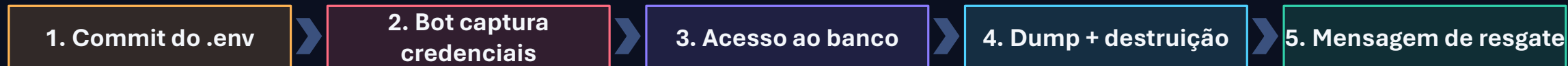
Quem assina é o servidor de autenticação. Quem valida não precisa conhecer a chave privada.

Conceitos

- Autenticidade: só o emissor legítimo consegue gerar assinatura válida.
- Não repúdio: a assinatura vincula o emissor ao token gerado.
- Integridade: alteração do payload invalida a assinatura.
- Confidencialidade exige JWE ou canal seguro adicional.

Segredos vazados no GitHub: o incidente nasce no commit, explode na produção

Linha do incidente



Quais ataques e vetores aparecem?

- Exposição de segredo por erro de processo/commit inseguro.
- Credenciais comprometidas e acesso não autorizado ao banco.
- Exfiltração de dados (dump) e destruição lógica das tabelas.
- Extorsão/ransom note como fase final do incidente.

Pilares afetados

- Confidencialidade: dump/copias dos dados.
- Integridade: tabelas apagadas.
- Disponibilidade: serviço perde a base original.

Automação que deveria existir

No commit

Hooks com Gitleaks, detect-secrets ou TruffleHog para bloquear push de secrets.

Na CI/CD

Secret scanning obrigatório, policy de falha no pipeline e integração com cofre de segredos.

Leia o cenário e pergunte: qual propriedade caiu? qual mecanismo explica o ataque?

Q1 • Cloud e CID

Resposta: D

- Bucket público de leitura compromete confidencialidade, não integridade.
- Multi-AZ + load balancer trabalham disponibilidade.
- Data breach por falha de aplicação fere confidencialidade.

Q2 • Assinatura digital

Resposta: A

- No modelo didático, o cliente assina o hash com sua chave privada.
- A API verifica com a chave pública correspondente.
- Isso sustenta autenticidade e não repúdio.

Q3 • Dupla extorsão

Resposta: B

- Ransomware simples prende acesso; dupla extorsão ainda ameaça vazamento.
- Logo, disponibilidade + confidencialidade ficam comprometidas.
- Se pedir tipo de artefato: ransomware, não worm nem spyware.

Seguindo...

Segurança ama precisão: hash não cifra, AES não assina e worm não precisa de clique.

Q4 • Dados em repouso

Resp. C

- Terabytes por dia pedem desempenho.
- AES/simétrica é a escolha correta para cifrar massa de dados.
- RSA não é para cifrar arquivos gigantes diretamente.

Q6 • Vírus x worm

Resp. C

- Vírus costuma depender de hospedeiro e ação do usuário.
- Worm é autônomo e se replica explorando rede.
- Trojan se disfarça; não é definido por replicar infinitamente na RAM.

Q5 • Login vulnerável

Resp. D

- Concatenação em query + OR '1'='1' = SQL Injection.
- XSS e CSRF são outras famílias de falha.
- A pista decisiva é o banco obedecendo ao input como comando.

Q7 • Senhas no banco

Resp. B

- Senha deve ser armazenada com hash resistente (Argon2/Bcrypt).
- Hash é unidirecional; aplicação compara hashes, não decripta senha.
- Criptografar senha com chave reversível aumenta risco operacional.

Controle de acesso, engenharia social e TLS reaparecem com frequência porque caem fácil e confundem rápido

Q8 • RBAC + logs

Resp. C

- RBAC restringe quem pode ver ou fazer algo: isso fortalece confidencialidade.
- Logs imutáveis sustentam rastreabilidade e não repúdio.

Q9 • GitHub falso

Resp. C

- E-mail + página clonada + captura de credenciais e 2FA = phishing/engenharia social.
- Não é brute force nem XSS; o alvo aqui é a vítima.

Q10 • TLS híbrido

Resp. B

- TLS autentica/troca segredo com criptografia assimétrica no começo.
- Depois usa chave de sessão simétrica para o tráfego porque precisa de desempenho.

Estude!

Objetivas 1-10

1-D

2-A

3-B

4-C

5-D

6-C

7-B

8-C

9-C

10-B

Relembrando

Malware

"WannaCry é um ransomware com comportamento de worm, pois se propaga autonomamente pela rede e indisponibiliza os dados por criptografia."

TLS

"O TLS usa criptografia híbrida: autentica e negocia a chave de sessão com mecanismos assimétricos; depois cifra o tráfego com criptografia simétrica."

SQLi

"A vulnerabilidade existe porque a aplicação concatena entrada não confiável à consulta SQL; a correção definitiva é o uso de consultas parametrizadas."

JWT

"JWT assinado garante integridade e autenticidade, mas não confidencialidade do payload sem criptografia adicional."

DevSecOps

"Segredos devem ser bloqueados antes do push e varridos na pipeline; se vazaram, a primeira ação é rotacionar as credenciais."

ATENÇÃO

Memorize...

1	Qual é o ativo?	Dado, identidade, serviço, segredo, token, sessão, infraestrutura?
2	Qual pilar caiu primeiro?	Confidencialidade, Integridade ou Disponibilidade?
3	Qual mecanismo técnico explica o ataque?	SQL Injection, phishing, worm, ransomware, assinatura, hash, TLS...
4	O que o atacante conseguiu fazer?	Ler, alterar, apagar, bloquear, fingir ser alguém ou escalar acesso?
5	Qual controle neutraliza ou reduz o risco?	Patch, MFA, RBAC, prepared statements, backup, segmentação, secret scanning...
6	Como escrever isso com verbo técnico?	Explorar, cifrar, verificar, assinar, parametrizar, autenticar, rotacionar, auditar.

**Resumo: assinar ≠ cifrar;
codificar Base64 ≠ proteger;
backup bom é backup
restaurável.**